

Types of files:

- .h → header files (interface) → just declare the functions.
- .c / .cpp → implementation → ...

DOMAIN + INTERFACE
(operations)

Planets Pb. (NASA)

Planet - name
- type
- distance

① ADD planet. (±)

② DISPLAY planets of given type.

↳ empty type ⇒ all planets displayed

③ DISPLAY planets within given distance fr. Earth

④ undo / redo

Modular programming

- separate interface (.h) from implementation (.cpp)
- hide implementation details (.cpp)
- header files (.h), source code files (.cpp)
- protect against multiple inclusions.

#ifndef

#define

#pragma once

Static allocation

typedef struct

```
{
  char name[50];
  char type[50];
  double distToEarth;
} Planet;
```

NOT
DYNAMIC

typedef struct

```
{
  Planet planets[100];
  int length;
} PlanetRepo;
```

vector of planets is
STATICALLY ALLOCATED.

int main()

```
{
  PlanetRepo repo = createRepo();
  Service serv = createService(&repo);
  UI ui = createUI(&serv);
  ...
  return 0;
}
```

→ all objects are statically allocated.

Dynamic allocation

→ memory allocated when we need it.

→ I am responsible with de-allocating it.

→ malloc, free (from stdlib.h)

→ objects must provide functions for creation
destruction

createPlanet ↙ return Planet object → static
↘ return pointer to a planet → dynamic

Planet* createPlanet(char* name, char* type, double distanceEarth)

{
 Planet* p = (Planet*) malloc(sizeof(Planet));
 ↙
pointer to Planet ↖ how much space (in bytes) occupies
the structure Planet.

reserve in heap the memory for one Planet
⇒ returns an address (void*)

void* → Planet* (this is called CASTING)

"treat this address as a pointer to Planet".

p → name = (char*) malloc(sizeof(char) * (strlen(name) + 1));

strcpy(p → name, name); // populate that memory

p → type = (char*) malloc(sizeof(char) * (strlen(type) + 1));

strcpy(p → type, type); // populate that memory.

? p → type = type;

return p;

}

void destroyPlanet(Planet* p)

{ free(p → name); } // free memory allocated for component fields

free(p → type);

free(p); → free memory allocated for the planet structure.

}

typedef struct

{ Planet* planets[100];

int length;

} PlanetRepo;

vector of planets will contain
pointers, not objects.

int main()

{ PlanetRepo* repo = createRepo();

Service* serv = createService(repo);

UI* ui = createUI(serv);

destroyUI(ui);

return 0; }

malloc → receives a size

↳ returns a pointer to the allocated memory.

malloc(sizeof(...))

Planet* createPlanet(....)

```
{
    Planet* p = malloc(sizeof(Planet));
    p->name = malloc((strlen(name)+1)*sizeof(char));
    ...
}
```

createPlanet("Jupiter", "gas giant", 1);

Planet p => p.name

Planet *p => (*p).name ⇔ p->name.

void test()

```
{
    Planet *p = CreatePlanet("earth", "planet", 0.0);
    assert(!strcmp(getType(p), "earth"));
    ...
}
```

In main → include just header files.

main:

```
#include <crtdbg.h>
#include "Planet.h"
int main(int argc, char **argv) {
    test();
    _CrtDump...
}
```

Repository.h

```
#pragma once
#include "Planet.h"
typedef struct {
    Planet** planets[10];
    int size;
} Repository;
```

Planet.h

```
typedef struct {
    char* name;
    char* type;
    double distToEarth;
} Planet;
```

```
Planet createPlanet(char* name,
char* type, double distToEarth);
void destroyPlanet(Planet* p);
char* getType(Planet* p);
Planet* createPlanet(...)
planet* p;
p->type;
getType(p)
void test() ...
```


Resize function:

Realloc

I. Capacity * 2

II. auxiliary pointer (aux)

III. copy

IV. free old memory

V. elems

↓

$v \rightarrow \text{capacity} = v \rightarrow \text{capacity} * 2;$

$\text{TElem} * \text{aux} = \text{malloc}(\text{sizeof}(\text{TElem}) * v \rightarrow \text{capacity});$

for (int i = 0; i < v → size; i++)

aux[i] = v → elems[i];

free(v → elems)

v → elems = aux;